PORTFOLIOHUB DOCUMENTATION

Full-Stack Backend Architecture

An end-to-end technical guide explaining models, routing, authentication, file uploading, error handling, and nested schemas.

AUTHOR
Abdelrhman Gom3a

ARCHITECTURE
MVC / RESTful API

STACK
Node.js · Express · MongoDB · EJS

DATE
May 2026

1. MVC Architecture & System Design

PortfolioHub is engineered using the classic **Model-View-Controller (MVC)** architectural design pattern. By decoupling data models, EJS template rendering, and business controller logic, the codebase achieves clean separation of concerns, scalability, and ease of maintainability.

Decoupled Components Flow

The system operates in a stateless environment powered by JSON Web Tokens. The request-response lifecycle traverses five major blocks:

Step 1: Router Mapping (REST Routes)

Incoming client queries hit the Express router handlers under `/api`. Routes parse variables, check URL formats, and bind directly to matching controller actions.

Step 2: Authentication Middleware (protect / optionalAuth)

Middleware parses JWT tokens from Authorization Bearer headers or HTTP-only cookies. Verified claims inject the `req.user` document context downstream. Unauthorized accesses throw instant 401 JSON halts.

Step 3: Controller Layer (Business Logic)

Controllers handle parameter sanitization, query database schemas, resolve logic conflicts, calculate averages (like star reviews), trigger notifications, and formatting responses.

Step 4: Mongoose Schema Model (Data Layer)

Interacts with MongoDB Atlas. Enforces schema validations, indexes, password hashing pre-save hooks, and populates document references.

Step 5: View Injection (Server-Side Rendering / EJS)

EJS layouts dynamically inject user statistics, explore cards, and initial dashboard states on the initial page loads, while client-side fetch wrappers update elements dynamically without reloads.

2. Directory Structure & File Map

Below is the complete blueprint of the backend directory tree, detailing folders and core scripts from root to nested leaves:

Path / File	Type	Architecture & Functional Purpose
`app.js`	Root File	Server entry point. Boots Express, registers middlewares (cors, cookie-parser), loads routes, connects mongoose, and hooks global errors.
`models/User.js`	Data Model	Defines individual/company schemas. Contains 30+ properties, password pre-save hash hook, and sub-schemas.
`models/Project.js`	Data Model	Portfolio project structure linked to user ownership via ObjectId.
`models/Comment.js`	Data Model	Rating/Review structure storing author, target profile, text comment, and numeric star value.
`models/Message.js`	Data Model	Stateless chat rooms saving conversation records, read/unread states, and group parameters.
`controllers/`	Directory	Houses all business logic. Functions are modularly grouped into `authController.js`, `userController.js`, `projectController.js`, `messageController.js`, `commentController.js`, and `adminController.js`.
`routes/`	Directory	Configures express endpoints, loading protect middlewares where authenticated access is enforced.

Path / File	Type	Architecture & Functional Purpose
`middleware/auth.js`	Middleware	Extracts JWT, validates session tokens, enforces block status checks, and blocks non-admins.

3. Database Design & Mongoose Schemas

PortfolioHub relies on MongoDB's document-oriented structure. To represent users, companies, and their activities, the system aggregates standard database fields and embedded sub-documents in a single master user collection.

Sub-documents Design in User.js

Rather than designing massive relational tables that require complex joins, sub-documents are nested directly inside the user model. This simplifies data reads for profiles and speeds up loads:

```
// Subdocument schema structures inside models/User.js
const ExperienceSchema = new mongoose.Schema({
    role:        { type: String, required: true },
    company:     { type: String, required: true },
    years:       { type: String },
    description: { type: String, default: '' }
}, { _id: true }); // _id: true allows sub-document target queries

const JobSchema = new mongoose.Schema({
    title:    { type: String, required: true },
    location: { type: String, required: true },
    link:     { type: String, required: true },
    type:     { type: String, enum: ['full-time', 'part-time', 'remote', 'inter
    postedAt: { type: Date, default: Date.now }
}, { _id: true });
```

Unified Schema Indexing

Performance in searching and exploring is accelerated by creating text indexes and single-field search fields inside the database:

- **Text Search Index**: Handles name, bio, and title simultaneously: `UserSchema.index({ name: 'text', bio: 'text', title: 'text' })`.
- **Role Indexing**: Speeds up filter requests on explore lists: `UserSchema.index({ role: 1 })`.

- **Email uniqueness**: Enforces strict unique login addresses.

4. Implementing Nested Profile CRUD Operations

The backend implements modular sub-document actions to add, update, and remove experience, education, branches, team, and jobs. Using Mongoose sub-document helpers (like `array.id(id)`), these actions query, alter, and save arrays within the master user record.

Example: Sub-document Update by ID

This controller action extracts the specific sub-document using the request parameters, updates individual fields, and performs atomic saves on the user document:

```
// From controllers/userController.js
const updateExperience = async (req, res) => {
    try {
        const { role, company, years, description } = req.body;
        const user = await User.findById(req.user._id);

        // Locate sub-document by ID
        const exp = user.experience.id(req.params.id);
        if (!exp) return res.status(404).json({ success: false, message: 'Exper

        // Update fields
        if (role) exp.role = role;
        if (company) exp.company = company;
        if (years) exp.years = years;
        if (description !== undefined) exp.description = description;

        await user.save({ validateBeforeSave: false });
        res.status(200).json({ success: true, experience: user.experience });
    } catch (err) {
        res.status(500).json({ success: false, message: 'Could not update exper
    }
};
```

Note on Company profile arrays: In `updateProfile`, nested lists (branches, jobs, team, timeline) are saved directly inside profile adjustments. A key mapping converts

timeline `description` inputs sent from the frontend client to the schema's Mongoose field name `desc` dynamically.

5. Authentication, JWT & Security

PortfolioHub runs a **stateless, token-based authentication session flow**. On successful registration or login, the backend issues a signed JSON Web Token (JWT) containing the user ID as its payload.

Double-Delivery Delivery System

To maximize browser compatibility, support mobile apps, and shield against CSRF, the JWT is delivered in two ways simultaneously:

1. **JSON Response Body**: Stored locally in browser `localStorage` by dynamic scripts for AJAX queries.
2. **HTTP-Only Cookie**: Handled automatically by the browser. Used by Express to render template files on page requests (SSR) without client-side scripts.

Token Validation Flow Diagram

Every protected route goes through the verification middleware:

> **1. Token Extraction**
>
> Middleware parses the Authorization header (`Bearer token`) or pulls the token from the request cookie.

> **2. JWT Decoding**
>
> Verifies the signature against `process.env.JWT_SECRET` . Decodes the user ID and verifies expiration.

> **3. Account Integrity Checks**
>
> Fetches the user document. Halts with 403 if `user.isBlocked` is set to true by an administrator.

> **4. Request Injection**

Attaches the user object to `req.user` and triggers the next controller action.

6. REST API Endpoint Reference (A to Z)

The following tables document every active endpoint on the PortfolioHub backend server:

Authentication & Identity

Method	Endpoint	Request/Response Payload Overview
POST	`/api/auth/register`	Registers user. Expects `{ name, email, password, role }`. Returns JWT + user object.
POST	`/api/auth/login`	Authenticates credentials. Expects `{ email, password }`. Delivers JWT.
POST	`/api/auth/logout`	Invalidates EJS cookies. Sets `isOnline: false` on database.
GET	`/api/auth/me`	Returns active user profile decoded from session JWT.

User Profiles & Nested CRUD

Method	Endpoint	Request/Response Payload Overview
GET	`/api/users`	Explore users. Accepts search filters `q`, `role`, `sort`, `page`.
PUT	`/api/users/profile`	Saves bio, name, social links, company details, or company nested arrays.
POST	`/api/users/experience`	Pushes experience object into user's profile. Returns updated array.
PUT	`/api/users/experience/:id`	Edits properties of experience matching the sub-document ID.
DELETE	`/api/users/experience/:id`	Removes experience object from sub-document array by ID.

Method	Endpoint	Request/Response Payload Overview
POST	`/api/users/education`	Pushes education object into user's profile. Returns updated array.
PUT	`/api/users/education/:id`	Edits properties of education matching the sub-document ID.
DELETE	`/api/users/education/:id`	Removes education object from sub-document array by ID.

6. REST API Endpoint Reference (Continued)

Connection Requests & Messaging

Method	Endpoint	Request/Response Payload Overview
POST	`/api/users/:id/request`	Sends network connection request. Expects `{ type: 'join' | 'invite' }`.
PUT	`/api/users/requests/:id`	Accepts or declines connection request. Expects `{ status: 'accepted' | 'declined' }`. If accepted, automatically hooks user to company team list.
DELETE	`/api/users/:companyId/team/:memberId`	Removes team member link from company. Authorized to company or member.
GET	`/api/messages`	Gets list of active conversation rooms.
POST	`/api/messages`	Sends text message. Creates conversation if first message.
GET	`/api/messages/:id`	Fetches messages in a room. Supports pagination.

Projects, Reviews & Uploads

Method	Endpoint	Request/Response Payload Overview
POST	`/api/projects`	Creates portfolio project. Expects title, desc, link, live demo, and image URL.
GET	`/api/projects/mine`	Returns list of projects owned by logged-in user.
DELETE	`/api/projects/:id`	Deletes project. Authorized to project owner or site admin.
POST	`/api/comments/:userId`	Leaves text comment and star rating on user profile. Pushes notification.
POST	`/api/upload`	Multer middleware processes single file binary. Returns uploads URL path.

7. File Upload & Data Sanitization Systems

File uploading is managed by the **Multer** middleware library inside `routes/upload.js`. The route is protected, ensuring only logged-in users can run uploads to prevent abuse.

Upload Constraints & Validations

- **Max File Size Limit**: Enforced at 50MB.
- **File Extensions Filtering**: The server performs a double verification on both mimetype and file extension:

```
const filetypes = /jpeg|jpg|png|gif|webp|pdf|doc|docx|ppt|pptx|xls|xlsx|txt|
```

- **Unique Target Naming**: A custom generator structure mitigates name collisions by prefixing date timestamps and random hashes to the original extension:

```
const uniqueSuffix = Date.now() + '-' + Math.round(Math.random() * 1E9);
cb(null, file.fieldname + '-' + uniqueSuffix + path.extname(file.originalnam
```

Server-Side Data Sanitization

To guard database entries and keep validation rules tight, the backend runs extensive checks:

Target	Sanitization / Regex Check	Purpose
Full Name	`/\d/.test(name)`	Blocks characters containing numbers. Enforces natural names.
Email Address	`/^[^s@]+@[^s@]+\.[^s@]+$/`	Verifies email formats are valid before executing database lookups.

Target	Sanitization / Regex Check	Purpose
Password Strength	`/^(?=.*[A-Za-z])(?=.*\d).{8,}$/`	Demands minimum 8 characters containing at least one letter and one number.

8. Error Handling & Innovation Mechanics

1. Global Express Error Handler

Express routes are wrapped inside try/catch blocks that automatically forward failures to a centralized error middleware at the bottom of `app.js`. This guarantees database drops or programming bugs never leak code stack trace logs to clients, instead returning formatted JSON responses:

```
// app.js — Global Error Middleware
app.use((err, req, res, next) => {
    console.error(`Global Error [${req.method} ${req.originalUrl}]:`, err.stack
    res.status(err.statusCode || 500).json({
        success: false,
        message: err.message || 'Internal Server Error'
    });
});
```

2. Profile Views Analytics

When loading `GET /api/users/:id`, the `getUserById` controller checks if the viewer is logged in and different from the profile owner. If true, it pushes a record containing user details and the view timestamp to the owner's `viewers` array, and sends a notification. A LinkedIn-style modal displays these analytic stats.

3. Automated Star Rating Average

When leaving ratings, the backend dynamically calculates user averages to avoid heavy processing delays during profile searches:

```
const currentTotal = (targetUser.rating || 0) * (targetUser.ratingCount || 0);
targetUser.ratingCount += 1;
```

```
targetUser.rating = (currentTotal + ratingVal) / targetUser.ratingCount;
await targetUser.save({ validateBeforeSave: false });
```